Отчет по реализованной модели ASR на базе архитектуры conformer

Докучаев В.О

23 октября 2026 г.

Содержание

1 Введение **2**

2 Архитектура **2**

3 Обучение модели **4**

4 Сложности с которыми я столкнулся, а также что не сработало **7**

1 Введение

Позволю себе перед написанием официального и строгого текста отчета маленькую преамбулу, которая расскажет о некоторых событиях предшествующих выполнению дз. В силу неведомых для меня до сих пор причин, я очень долго не мог начать делать это дз Хотя честно пытался, был какой-то мандраж, который сводил на нет все мои начинания. В итоге, к выполнению дз я подошел за несколько дней до дедлайна, имея за спиной лишь маленькие наработки. И вот в момент, когда больше тянуть нельзя, я, мой нервный тик на правом глазу, острый отит на левом ухе (что довольно иронично, для дз по распознаванию речи), вагон и маленькая загонов, начали работать сей отнюдь недружной компанией. Следующее дз, начну делать сразу же как оно дропнется, а сейчас довольно грустненько

Как уже было сказано в ранее, мною была реализована модель на базе архитектуры Conformer и чье устройство будет разобрано подробнее ниже. Также для оснвной модели был реализован BPE + LM энкодер. Обучение производилось на домашней RTX 5060 с 8 ГБ видеопамяти на основе датасета LibriSpeech train-clean-100 с 5 аугментациями. Удалось получить такие результаты:

Таблица 1: Сравнение качества моделей Conformer-CTC на LibriSpeech

		test-clean		test-other	
Токенизация	Декодирование	WER, %	CER, %	WER, %	CER, %
Character	Greedy	22.775	7.158	48.159	20.078
Character	Beam	22.299	7.009	47.655	19.798
Character	Beam + LM	17.586	6.083	42.751	18.794
BPE	Greedy	18.025	6.944	41.650	20.286
BPE	Beam	17.772	6.827	41.432	20.106
BPE	Beam + LM	14.075	5.618	37.056	18.377
BPE (14 epochs)	Beam + LM	**13.843**	**5.507**	**36.880**	**18.257**

Все модели, кроме последней, обучались 10 эпох. Последняя модель дополнительно обучалась 4 эпохи. Beam width = 8. LM: KenLM 3-gram, обученная на LibriSpeech train-960. Размеры тестовых выборок: test-clean — 2620 записей, test-other — 2939 записей.

Обучение заняло порядка 2 часов для моделей на 10 эпох. И примерно час для последующего файнтюнинга

Таблица 2: Ход обучения Conformer-BPE на LibriSpeech

	CTC loss		dev-clean		
Эпоха	Train	Val	WER, %	CER, %	Время, мин:с
1	3.562	1.900	58.485	29.131	12:56
2	1.462	1.205	41.362	18.099	13:46
3	1.046	0.964	33.866	14.173	13:48
4	0.828	0.835	29.170	11.944	14:12
5	0.671	0.731	25.269	10.240	13:28
6	0.541	0.629	22.249	8.810	13:34
7	0.437	0.569	19.942	7.806	18:39
8	0.353	0.539	18.549	7.183	13:23
9	0.294	0.521	17.759	6.848	13:41
10	0.270	0.518	17.678	6.779	14:19

2 Архитектура

За основу акустической модели взята архитектура Conformer из статьи Gulati et al., «Conformer: Convolution-augmented Transformer for Speech Recognition» (2020). Она объединяет многоголовое внимание с относительным позиционным кодированием и свёрточный модуль. В проекте используется функция потерь CTC. Общая схема показана на рис. 1.

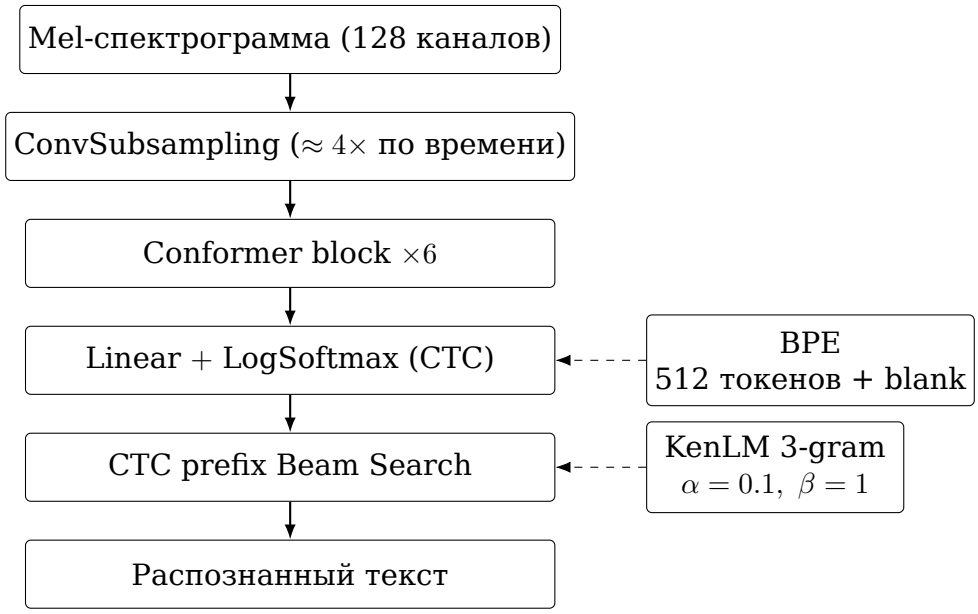


Рис. 1: Реализованная система Conformer-CTC с BPE и языковой моделью при декодировании.

На вход модели подаётся Mel-спектрограмма с 128 каналами. Два свёрточных слоя уменьшают длину последовательности примерно в четыре раза, после чего она проходит через шесть блоков Conformer.

Каждый блок содержит два модуля Feed Forward, многоголовое внимание и свёрточный модуль. Для модулей Feed Forward применяются остаточные соединения с коэффициентом 0.5, а после второго модуля выполняется LayerNorm. Линейный слой и LogSoftmax формируют логарифмы вероятностей CTC-токенов.

Основная модель реализована в src/model/conformer.py, а её блоки и вспомогательные компоненты находятся в src/model/conformer_modules.py.

Для представления текста использовалась токенизация BPE из библиотеки Hugging Face tokenizers. Словарь содержит 512 токенов; ещё один выходной класс выделен для CTC blank. Токенизатор обучался на транскрипциях LibriSpeech train-clean-100, train-clean-360 и train-other-500. Реализация BPE находится в src/text_encoder/bpe.py, а скрипт обучения токенизатора и языковой модели доступен в script/train_text_models.py.

При инференсе применялся собственный CTC prefix beam search с шириной луча 8 и языковой моделью KenLM 3-gram. Для ранжирования префиксов использовалась оценка

$$S(y) = \log P_{\text{CTC}}(y \mid x) + \alpha \log P_{\text{LM}}(y) + \beta N_{\text{words}}(y), \qquad \alpha = 0.1, \quad \beta = 1.0. \tag{1}$$

Реализация поиска находится в src/text_encoder/beam_search.py, а интеграция KenLM реализована в src/text_encoder/lm.py. На контрольной точке десятой эпохи подключение KenLM к BPE с Beam Search снизило WER с 17.772% до 14.075% на test-clean и с 41.432% до 37.056% на test-other.

Модель обучалась на основе конфига configs/clean100_aug10.yaml

3 Обучение модели

При обучении применялись пять аугментаций:

- Gain - случайное изменение громкости в диапазоне ± 6 дБ с вероятностью $p = 0.5$.
- Pitch Shift - изменение высоты тона в пределах ± 1 полутона с вероятностью $p = 0.3$.
- Time Stretch - изменение скорости речи с коэффициентом 0.9–1.1 без изменения высоты тона, с вероятностью $p = 0.3$.
- Time Masking - маскирование до 30 временных отсчётов спектрограммы (не более 10% длины) с вероятностью $p = 0.5$.
- Frequency Masking - маскирование до 12 частотных каналов спектрограммы с вероятностью $p = 0.5$.

Они выбирались по принципу: если я хочу, чтобы моя модель распознавала разную речь, то надо изменить аудио те, что переводят голос человека в голос другого человека, а другие люди могут говорить медленнее или быстрее, их голос может быть выше или ниже. Для аугментаций спектограммы я выбирал те, что вносят для модели контролируемый хаос, чтобы подготовить ее к зашумленным и несовершенным данным реальности.

Для обучения использовалась модель Conformer-CTC с 6 блоками, размерностью скрытого представления 256, 4 головами внимания, размерностью FFN 1024 и ядром свёртки 31. Dropout составлял 0.1. Модели обучались на LibriSpeech train-clean-100 в течение 10 эпох с размером батча 4. Для оптимизации использовался AdamW с weight decay 0.01 и планировщиком OneCycleLR, максимальная скорость обучения составляла $6 \cdot 10^{-4}$, доля разогрева — 0.1, ограничение нормы градиента — 5.

Аудиозаписи преобразовывались в Mel-спектрограммы с 128 фильтрами, размером FFT 512, длиной окна 400 и шагом 160 при частоте дискретизации 16 кГц. При обучении применялись аугментации Gain (±6 дБ, $p = 0.5$), Pitch Shift (±1 полутон, $p = 0.3$), Time Stretch (0.9–1.1, $p = 0.3$), Time Masking (до 30 временных отсчётов, $p = 0.5$) и Frequency Masking (до 12 частотных каналов, $p = 0.5$). Сравнивались две конфигурации с одинаковой архитектурой и параметрами обучения: символьная токенизация с 28 выходными классами и BPE с 513 классами, включая CTC blank. Для валидации использовалась LibriSpeech dev-clean, декодирование выполнялось жадным методом.

Опытным путем на основе нескольких запусках удалось установить, что граница эффективности модели упирается в 17% wer. В первые 10 эпох она довольно активно снижала ошибку, но при файнтюнинге изменения стали в районе 0.1%

Также удалось установить, важность параметров для KenLM. В одном из экспериментов было установлено, Для декодирования с KenLM использовались два основных параметра: α и β. Параметр α определяет вес языковой модели относительно акустической, а β задаёт дополнительное вознаграждение за каждое завершённое слово, компенсируя склонность CTC к коротким гипотезам. Эти параметры настраивались на подвыборке dev-clean. Для Pruned 3-gram увеличение α с 0.1 до 0.5 и β с 1.0 до 1.5 снизило WER с 18.37% до 13.57% на test-clean и с 43.56% до 36.71% на test-other. Для Full 4-gram оптимальными на валидации оказались $\alpha = 0.7$ и $\beta = 1.5$, что дало WER 12.37% и 35.67% соответственно. Таким образом, подбор весов декодера существенно повлиял на качество распознавания даже без изменения самой акустической модели.

В финальной версии датасета использовалась объединенный текст-

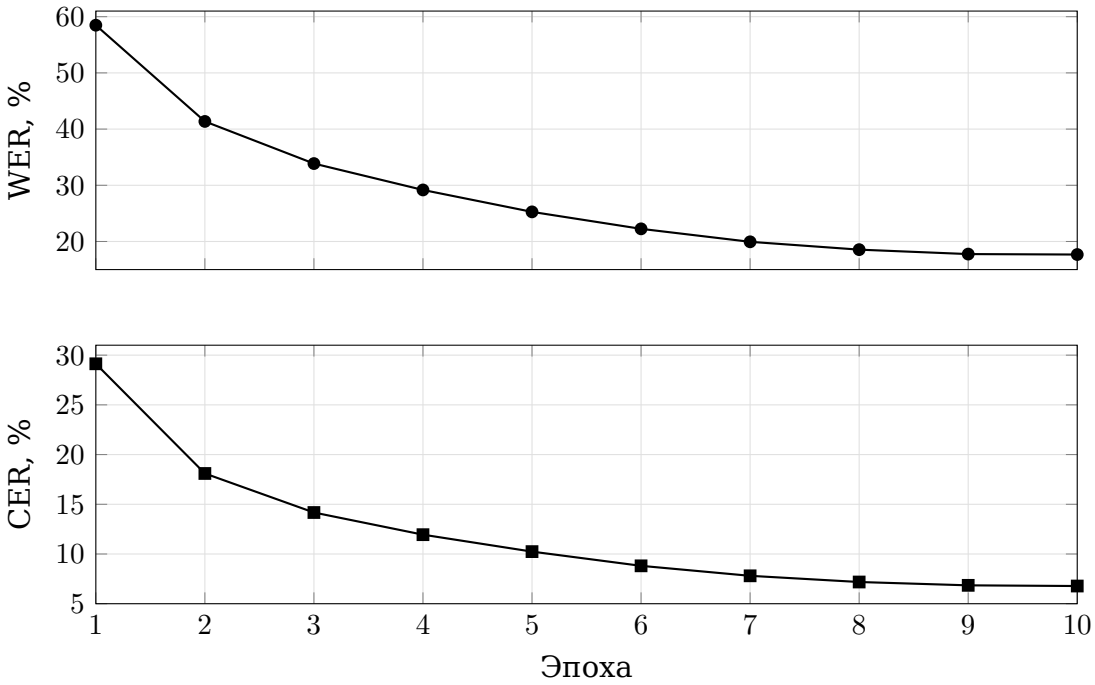


Рис. 2: Динамика WER и CER на LibriSpeech dev-clean при обучении Conformer-BPE.

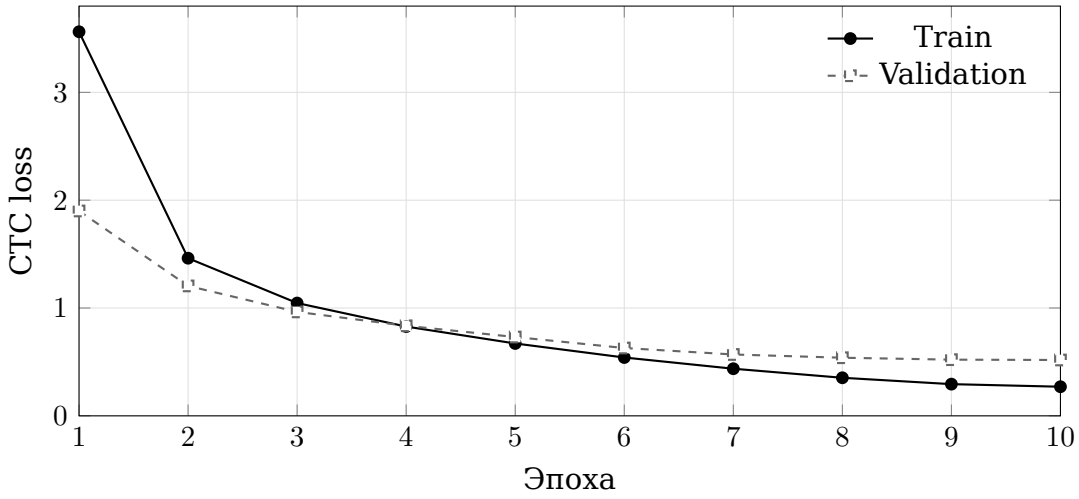


Рис. 3: CTC loss на обучающей и валидационной выборках.

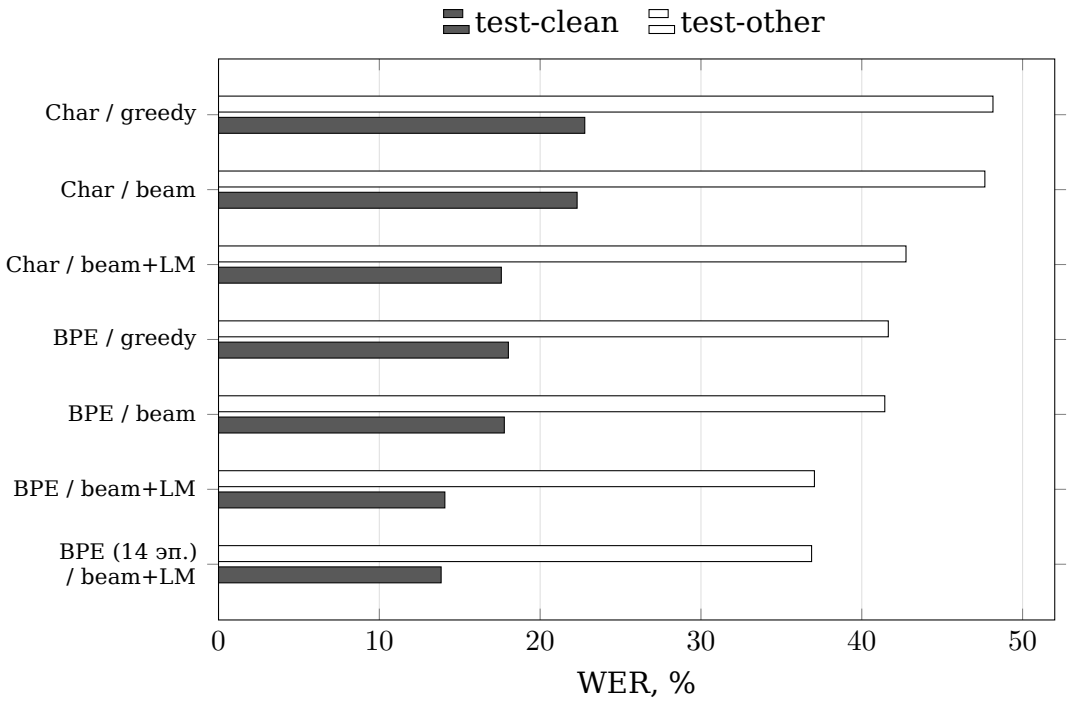


Рис. 4: Сравнение WER на тестовых выборках при различных токенизаторах и способах декодирования.

вый корпус из датасетов train-clean-100, train-clean-360 и train-other-500

4 Сложности с которыми я столкнулся, а также что не сработало

Прежде всего это сложности связанные с тайм менджемнтом. А также возникли сложности с развертыванием модели в облаке. Оказалось, что библиотека kenlm, которую я использовал для части связанной с LM декодером не хотела ставиться на kaggle из-за конфликта версий с окружением, поэтому использовалась эксперементальная версия пакета